The `train` function trains a single neural network using stored experiences in an experience replay memory buffer. It has the following inputs and outputs:

- Inputs:
  - `batch_size` (`int`)
  - `replay`, list of tuples `(obs_1_small, acts_1,rewards1,act_means1,qnext1)`
  - `param` (`vector`) neural network parameter vector
  - `layers` (`list`) contains shape of neural network layers
  - `J` (`int`) number of agents on this team
  - `gamma` (`float` in `[0,1]`) discount factor
  - `lr` (`float`) learning rate for SGD
- Returns:
  - `loss` (`float`)

We've now set up the environment, set up the agents for the two teams, and defined several functions to let us train the two DQNs we're using for mean field Q-learning. Now we get into the main loop of game play. Be warned, there is a lot of code in the next few listings, but most of it is just boilerplate and isn't critical for understanding the overall algorithm.

Let's first set up our preliminary data structures, like the replay buffers. We will need separate replay buffers for team 1 and team 2. In fact, we will need almost everything separate for team 1 and team 2.

Listing 9.19. Initializing the actions

```
N1 = env.get_num(team1)                                          1
N2 = env.get_num(team2)
step_ct = 0
acts_1 = torch.randint(low=0,high=act_space,size=(N1,))          2
acts_2 = torch.randint(low=0,high=act_space,size=(N2,))

replay1 = deque(maxlen=replay_size)                              3
replay2 = deque(maxlen=replay_size)

qnext1 = torch.zeros(N1)                                         4
```